

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Universidad Tecnológica Nacional (UTN)

Programación 3

Trabajo Práctico Integrador

Food Store

Sistema de Gestión de Pedidos de Comida

Estudiante: Niemiec Maximiliano

Fecha de entrega: 02/07/2026

Índice

Índice	2
1. Introducción	3
2. Marco Teórico	3
2.1 Backend	3
2.2 Frontend	3
2.3 Conceptos y patrones de diseño aplicados	3
3. Decisiones Técnicas y Arquitectura	4
3.1 Estructura general	4
3.2 Entidad Base y Repositorio Base	4
3.3 Patrón DTO y actualizaciones parciales	4
3.4 Manejo global de excepciones	4
3.5 Seguridad de contraseñas	4
3.6 Transaccionalidad en la creación de pedidos	5
3.7 Modelo de dominio	5
3.8 Carga inicial de datos	5
3.9 Integración Frontend–Backend	5
3.10 Capturas de las interfaces principales	5
4. Dificultades y Soluciones	12
4.1 Consistencia del stock ante errores	12
4.2 Actualizaciones parciales sin sobrescribir datos	12
4.3 Respuestas de error inconsistentes	12
5. Bibliografía / Webgrafía	12

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Integrador de la materia Programación 3, correspondiente a la Tecnicatura Universitaria en Programación a Distancia (UTN). El proyecto consiste en el desarrollo de Food Store, una aplicación web full stack para la gestión de pedidos de un negocio de comidas, compuesta por una API REST desarrollada en Spring Boot y un cliente web desarrollado en TypeScript con Vite.

El desarrollo se guió por un backlog de 29 historias de usuario, organizadas en cinco épicas: Gestión de Categorías, Gestión de Usuarios, Gestión de Productos, Gestión de Pedidos e Infraestructura y Arquitectura. Cada historia fue implementada respetando sus criterios de aceptación, reglas de negocio y tareas técnicas específicas.

2. Marco Teórico

A continuación se describen brevemente las tecnologías y conceptos aplicados en el desarrollo del proyecto.

2.1 Backend

- Spring Boot 3.2.5 (Java 21): framework que simplifica la configuración de aplicaciones Java empresariales, provee un servidor embebido y un modelo de inyección de dependencias.
- Spring Data JPA / Hibernate: capa de persistencia orientada a objetos (ORM) que mapea las entidades del dominio a tablas de la base de datos, evitando escribir SQL manual para las operaciones CRUD.
- Jakarta Bean Validation: conjunto de anotaciones (@NotBlank, @Size, @Min, @DecimalMin, etc.) que permiten declarar reglas de validación directamente sobre los DTO de entrada.
- Spring Security Crypto (BCrypt): algoritmo de hashing con "salt" aleatorio utilizado para almacenar las contraseñas de forma irreversible, evitando guardar texto plano.
- SpringDoc OpenAPI (Swagger): genera automáticamente documentación interactiva de los endpoints de la API a partir del código.
- Base de datos H2: motor relacional liviano basado en archivo, utilizado para el entorno de desarrollo y evaluación del proyecto.

2.2 Frontend

- TypeScript: superset de JavaScript que añade tipado estático, reduciendo errores en tiempo de desarrollo.
- Vite: herramienta de build y servidor de desarrollo con recarga en caliente (HMR), utilizada para compilar y servir el cliente web.
- localStorage: API del navegador utilizada para persistir el carrito de compras y los datos de sesión del usuario en el cliente (con fines educativos, sin tokens de seguridad reales).

2.3 Conceptos y patrones de diseño aplicados

- Arquitectura en capas (Controller → Service → Repository): separa la exposición HTTP, la lógica de negocio y el acceso a datos.
- Patrón DTO (Data Transfer Object): objetos específicos para la entrada (Create/Edit) y salida (Dto) de cada entidad.

- Soft Delete (borrado lógico): las entidades no se eliminan físicamente, sino que se marcan con un campo booleano eliminado, preservando la integridad histórica.
- Optimistic Locking: control de concurrencia mediante un campo version, que evita que dos actualizaciones simultáneas sobrescriban datos sin detectarlo.
- Manejo centralizado de excepciones (@RestControllerAdvice): unifica el formato de las respuestas de error de toda la API.

3. Decisiones Técnicas y Arquitectura

3.1 Estructura general

El backend se organizó siguiendo una arquitectura en capas clásica, separando responsabilidades en los paquetes: model (entidades JPA), repository (acceso a datos), service (lógica de negocio y reglas), controller (endpoints REST), dto (contratos de entrada/salida), exception (manejo de errores) y config (seguridad, CORS, carga inicial de datos).

3.2 Entidad Base y Repositorio Base

Se implementó una clase abstracta Base (@MappedSuperclass) de la que heredan todas las entidades del dominio (Categoria, Usuario, Producto, Pedido, DetallePedido). Esta clase centraliza el identificador autogenerated, el campo eliminado para soft delete, los timestamps de auditoría (createdAt / updatedAt) y el campo version para optimistic locking. De esta forma se evita duplicar estos campos en cada entidad.

De manera análoga, se creó una interfaz genérica BaseRepository<E extends Base, ID>, de la que heredan todos los repositorios concretos. Provee métodos por defecto como findByIdOrThrow() (que lanza ResourceNotFoundException si el registro no existe o fue eliminado) y softDeleteById() (que marca el registro como eliminado en lugar de borrarlo físicamente), evitando repetir esta lógica en cada servicio.

3.3 Patrón DTO y actualizaciones parciales

Cada entidad cuenta con tres tipos de DTO: uno de creación (Create, con validaciones estrictas de campos obligatorios), uno de edición (Edit, con campos opcionales para permitir actualizaciones parciales tipo PATCH/PUT) y uno de salida (Dto, que expone únicamente los campos seguros de mostrar). Este patrón evita, por ejemplo, que el campo contraseña del usuario viaje en las respuestas de la API. Los DTO de edición implementan un método applyTo() que aplica sobre la entidad solamente los campos no nulos recibidos, preservando el resto de los valores existentes.

3.4 Manejo global de excepciones

Se centralizó el tratamiento de errores en GlobalExceptionHandler (@RestControllerAdvice), que traduce cada tipo de excepción a un código HTTP y a un formato de respuesta consistente: ResourceNotFoundException → 404, BusinessException (violaciones de reglas de negocio, como stock insuficiente o email duplicado) → 400, errores de validación de Bean Validation → 400 con el detalle de cada campo, IllegalStateException → 409 y cualquier otra excepción no controlada → 500, registrando el error en el log del servidor.

3.5 Seguridad de contraseñas

Las contraseñas de los usuarios se encriptan mediante BCryptPasswordEncoder (Spring Security Crypto) antes de persistirse, y se verifican mediante comparación de hash en el login. El proyecto no implementa autenticación basada en tokens (JWT); el frontend conserva los datos de la sesión en localStorage, tal como establece el alcance educativo del trabajo.

3.6 Transaccionalidad en la creación de pedidos

La creación de un pedido (HU-017) es la operación más sensible del sistema, ya que involucra validar la existencia y disponibilidad de cada producto, verificar stock suficiente, descontar stock y calcular el total. Todo el proceso se ejecuta dentro de un método anotado con @Transactional, de forma que si cualquier validación falla a mitad del recorrido (por ejemplo, el segundo producto del carrito no tiene stock), Spring revierte automáticamente (rollback) los cambios ya aplicados sobre los productos anteriores, evitando dejar la base de datos en un estado inconsistente.

3.7 Modelo de dominio

El modelo respeta el diagrama UML propuesto en la consigna: Pedido implementa la interfaz Calculable (con el método calcularTotal()) y expone los métodos addDetallePedido(), findDetallePedidoByProducto() y deleteDetallePedidoByProducto() para gestionar su colección de DetallePedido. Las relaciones entre entidades (Usuario–Pedido, Pedido–DetallePedido, DetallePedido–Producto, Producto–Categoria) se modelaron con las anotaciones JPA correspondientes (@OneToMany, @ManyToOne).

3.8 Carga inicial de datos

Al iniciar la aplicación, el componente UserLoad (CommandLineRunner) crea dos usuarios de prueba (un administrador y un cliente) si la base de datos está vacía, con sus contraseñas ya encriptadas. En paralelo, import.sql carga categorías y productos de ejemplo, permitiendo evaluar el sistema sin necesidad de cargar datos manualmente.

3.9 Integración Frontend–Backend

El frontend se organizó en páginas según el rol del usuario: store (catálogo, detalle de producto, carrito) y client (historial de pedidos) para el rol USUARIO, y admin (dashboard, categorías, productos, pedidos) para el rol ADMIN. La comunicación con la API se realiza mediante fetch a los endpoints REST documentados, habilitados mediante configuración de CORS para el origen del cliente.

3.10 Capturas de las interfaces principales

A continuación se incluyen capturas de las pantallas principales del sistema, tanto del lado del cliente como del panel de administración.





Iniciar Sesión - Food Store

Ingresá tus credenciales de acceso.

Correo Electrónico *

Contraseña *

Ingresar al Sistema



*Credenciales Cliente: cliente@food.com / cliente123 Credenciales
Admin: admin@food.com / admin123*

¿No tiene cuenta? Regístrese acá



Registro de Cliente - Food Store

Cree su cuenta para empezar a comprar. Solo se permite el registro de clientes.

Nombre *

Apellido *

Correo Electrónico *

Celular

Contraseña * (Mínimo 6 caracteres)

Registrarse y Entrar

[¿Ya tiene cuenta? Inicie sesión](#)

Sesión: Juan Perez (USUARIO)

Mis Pedidos

Tienda Store

Cerrar Sesión

Nombre: A-Z

 Carrito ⁰



Hamburguesa Clasica
Carne 150g, lechuga y tomate

\$ 4500

Disponible



Hamburguesa Doble
Doble carne con cheddar y panceta

\$ 6200

Disponible



Pizza Mozzarella
Mozzarella, salsa de tomate y oregano

\$ 3800

Disponible

Se encontraron 3 productos disponibles

Categorías

Mostrar Todo

Hamburguesas

Pizzas



ID Producto: # 1

Hamburguesa Clasica

Carne 150g, lechuga y tomate

\$ 4500

Disponible! (50 u.)

Cantidad:

 **Añadir al Carrito**

Detalle de tu Carrito de Compra

Revise los ítems y las cantidades antes de proceder al pago transaccional.



Hamburguesa Clasica
\$ 4500 c/u

- 1 +

\$ 4500



 Vaciar Carrito Completo

Resumen de Orden

Subtotal Neto: \$ 4500

TOTAL: **\$ 4500**

 Forma de Pago *

 Efectivo en Caja ▼

 Teléfono de Contacto *

Ej: 3513445566

Confirmar Pedido

Seguir Comprando

localhost:5173 dice

¡Pedido confirmado con éxito! Tu orden ya está ingresada en la cocina. Nos comunicaremos al número 35623562 ante cualquier novedad.

Aceptar



Mis Pedidos Realizados

Hacé clic sobre cualquier tarjeta para inspeccionar el detalle completo y el estado de entrega.

Orden # 1

Pendiente



Emitido: 02/07/2026 17:30



Contenido: 1x Hamburguesa Clasica

Total abonado:

\$ 4500

Sesión: Ana Martinez (ADMIN)

Panel Admin

Mis Pedidos

Tienda Store

Cerrar Sesión

Panel de Control Gerencial y Auditoría

Total Categorías

2

Grupos de menú activos

Total Productos

3

Ítems registrados en catálogo

Total Pedidos

1

Órdenes globales históricas

Items Disponibles

3

Con unidades operativas en cocina

Desglose Operativo del Establecimiento

Auditoría de Catálogo

Categorías Activas: 2 (0 inactivas).

Productos Activos (Disponibles): 3 ítems con stock.

Productos Inactivos (Sin Stock): 0 ítems a reponer.

Pedidos por Estado

Pendientes de aprobación: 1 órdenes.

En preparación (Confirmados): 0 órdenes.

Entregados (Terminados): 0 órdenes finalizadas.

Cancelados: 0 órdenes.

Administración de Categorías de Menú

[+ Nueva Categoría](#)

Defina y edite las familias de platos que se muestran en el catálogo de clientes.

[Volver al Dashboard](#)

ID	Vista previa	Nombre de la Categoría	Descripción Operativa	Acciones
# 1		Hamburguesas	Hamburguesas artesanales premium	 
# 2		Pizzas	Pizzas al horno de barro	 

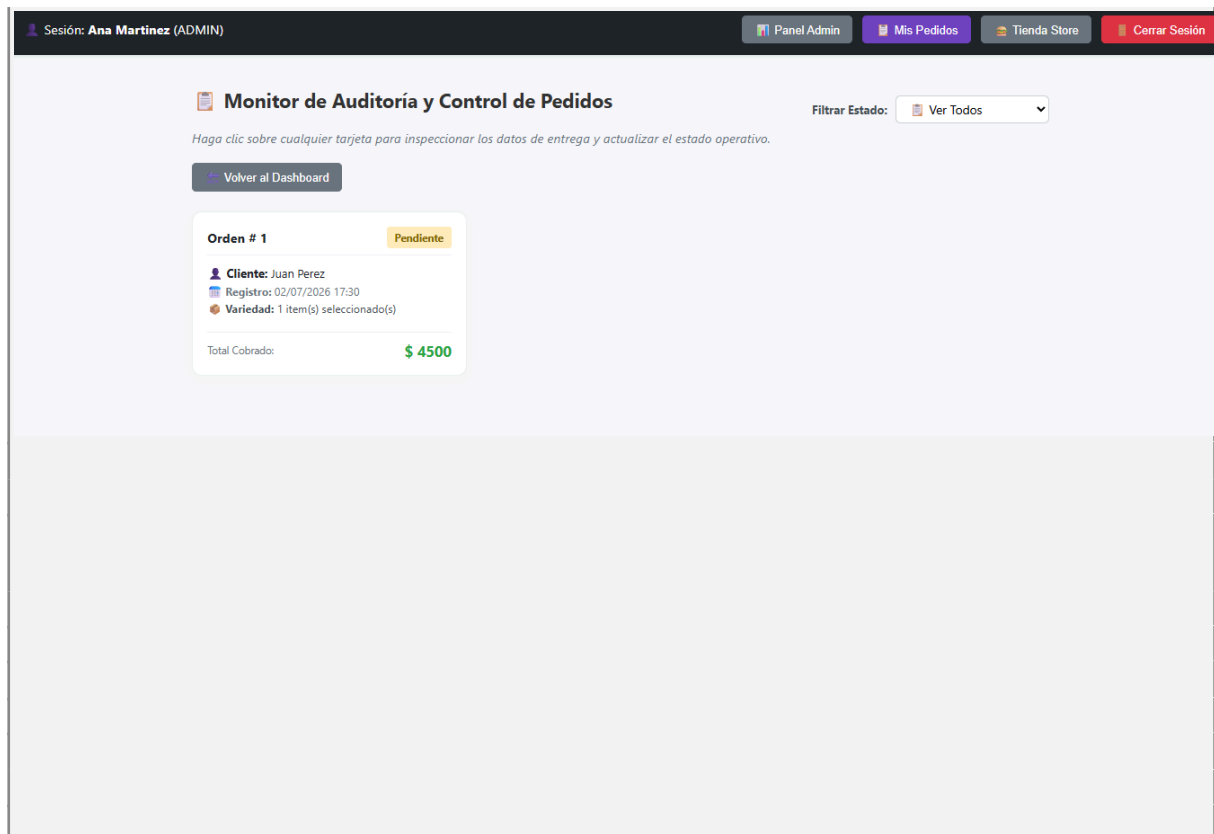
Inventario Operativo de Productos (ABM)

[+ Nuevo Producto](#)

Modifique precios, controle el stock de seguridad y actualice la disponibilidad en mostrador.

[Volver al Dashboard](#)

ID	Vista previa	Nombre del Plato	Descripción Corta	Precio	Categoría	Stock	Estado	Acciones
# 1		Hamburguesa Clásica	Carne 150g, lechuga y tomate	\$ 4500	Hamburguesas	49 u.	Disponible	 
# 2		Hamburguesa Doble	Doble carne con cheddar y panceta	\$ 6200	Hamburguesas	40 u.	Disponible	 
# 3		Pizza Mozzarella	Mozzarella, salsa de tomate y oregano	\$ 3800	Pizzas	25 u.	Disponible	 



4. Dificultades y Soluciones

4.1 Consistencia del stock ante errores

Durante la creación de un pedido con varios productos, era necesario garantizar que, si fallaba la validación de stock de un producto, no quedaran descontadas las unidades de los productos ya procesados anteriormente en el mismo pedido. Se resolvió delegando el control de atomicidad en la anotación `@Transactional` de Spring, que revierte automáticamente todos los cambios ante cualquier excepción no capturada dentro del método.

4.2 Actualizaciones parciales sin sobrescribir datos

Los endpoints PUT debían permitir actualizar solo algunos campos sin pisar el resto con valores nulos. Se resolvió con el patrón `applyTo()` en los DTO de edición, que actualiza únicamente los campos distintos de null recibidos en el cuerpo de la petición.

4.3 Respuestas de error inconsistentes

En una primera etapa del desarrollo, distintas violaciones de reglas de negocio (email duplicado, stock insuficiente, producto no disponible) se lanzaban como excepciones genéricas de Java, lo que provocaba que la API respondiera con código 500 en lugar de 400 en esos escenarios. Se solucionó introduciendo una jerarquía de excepciones propia (`BusinessException`, `ResourceNotFoundException`) capturada de forma centralizada por `GlobalExceptionHandler`.

5. Bibliografía / Webgrafía

- [Spring Boot Reference Documentation](#)
- [Spring Data JPA Reference Documentation](#)
- [Jakarta Bean Validation](#)
- [Spring Security – Password Storage](#)
- [SpringDoc OpenAPI](#)
- [H2 Database Engine](#)
- [Vite – Documentación oficial](#)
- [TypeScript – Documentación oficial](#)
- **Cátedra de Programación III.** (2026). *Apuntes teóricos y guías de trabajos prácticos de la asignatura Programación III.* Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional (UTN).